

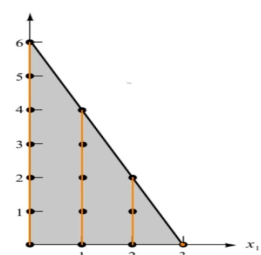
## EGÉSZ ÉRTÉKŰ FELADATOK ÉS JELLEMZŐIK

- bizonyos LP feladatoknál ki kell kötni, hogy a megoldás során, bizonyos változók csak egészek lehessenek
  - pl: cég gyárt fából vonatokat és katonákat, akkor nem lehetséges, nem logikus 2.5 vonatot gyártani vagy bináris döntés sem lehet tört (építünk gyárat vagy sem)
- szimplex algoritmusban nincs semmilyen garancia, hogy a végeredmény egész lesz és amennyiben korlátozzuk a lehetséges megoldásokat csak az egészek körébe, akkor ez egy jóval nehezebb feladathoz vezet
- **Egészértékű programozási feladat - ILP (Integer Linear Programming)**
  - akkor beszélünk ILP-ről, ha az LP feladatban a változók (vagy csak egy részük) csak nemnegatív egész értéket vehetnek fel
- Vegyes egészértékű programozási feladat (MIP - Mixed Integer Programming)
  - olyan LP feladat, amelyikben csak néhány (nem az összes) változóra követeljük az egészértékűséget (fenti példa esetén  $x_1, x_2$  egész feltétel helyett mondjuk csak  $x_1$  egész-t írunk)
- 0-1 egészértékű programozási feladat (0-1 IP - 0-1 Integer Programming)
  - olyan IP (tisztá egészértékű programozási feladat) ahol minden változó bináris, azaz csak 0 vagy 1 lehet az értékük
- Tiszta egészértékű programozási feladat (IP - Integer Programming)

$$\begin{aligned} \max \quad & z = 3x_1 + 2x_2 \\ \text{f.h.} \quad & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0 \end{aligned}$$

$x_1, x_2$  egész

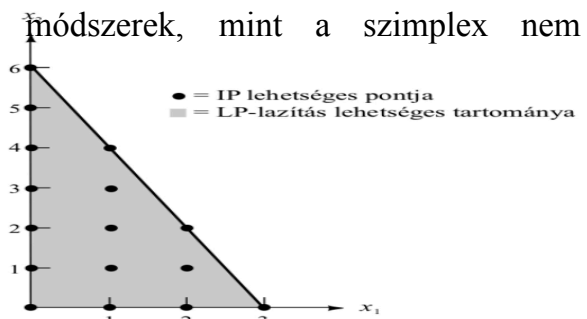
- olyan LP feladat, amelyikben mindegyik döntési változó egész értékű
- NP-nehezek (polinomiális időben nem megoldhatók)
- a lehetséges megoldások halmaza nem konvex poliéder, hanem diszkrét pontrács, ezért csúcspont-kereső módszerek, mint a szimplex nem alkalmazhatók közvetlenül



A) LP (szürke háromszög)

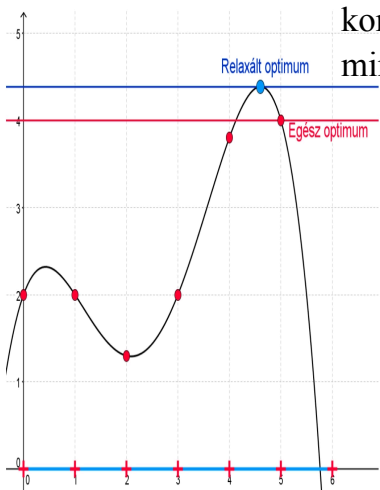
B) Vegyes egészértékű programozási feladat (sárga szakaszok)

C) IP, azaz tiszta egészértékű programozási feladat (fekete pontok)



- Szimplex algoritmus nem alkalmazható az IP feladatokra, mert a lehetséges megoldások halmaza ezeknél nem konvex poliéder, erre megoldás az **LP-lazítás** azaz **LP-relaxálás**
  - relaxálás = elhagyás, azaz elhagyjuk a változókra tett minden egészértékűségi vagy 0-1 (bináris) feltételeket (pl: " $x_1, x_2$  egész" feltétel elhagyása)

- a relaxált LP feladatot megoldjuk simán mint egy LP feladatot, ez egy felső korlátot ad a maximumra (minimalizálás esetén pedig egy alsó korlátot a minimumra)



- ezért a **relaxált feladat optima** **sosem rosszabb mint az egészértékű feladat optima**, ezért ha egész értéket kaptunk a relaxált feladatra akkor megkaptuk az eredeti feladat optimális megoldását
  - maximalizálás esetén sosem kisebb, minimalizálás során sosem nagyobb mint az egészértékű feladat optima
- viszont az eredménye gyakran tört, ezért az eredeti feladatok nem oldhatóak meg a relaxált feladat optimális megoldásának kerekítésével, mert ez vagy nem lehetséges vagy mert nem ad optimális megoldást
  - **nem-fizibilitás (nem lehetséges megoldás):** ha a relaxált megoldást pl: 3.8 lekerekítjük 3-ra vagy felkerekítjük 4-re, akkor előfordulhat, hogy a kapott pont kívül esik a megoldások halmazán
  - **szuboptimilitás:** a kerekített pont nagyon messze lehet a valódi egészértékű optimumtól

### Korlátozás és szétválasztás (Branch and Bound - B&B)

- ez egy ILP-ket (egész értékű programozási feladatokat) megoldó stratégia, lényege a korlátozás és a szétválasztás (oszd meg és uralkodj stratégia)
  - szisztematikus keresés egy fa struktúrában (keresési fa)
  - oszd meg és uralkododj: a feladatot kisebb rész feladatokra bontja (branch), de a felesleges ágakat próbáljuk minnél korábban levágni (korlátozás, bound), hogy ne kelljen az összes lehetőséget bejárni
    - levágás úgy működik, hogy egy-egy csúcshoz érve kiszámítjuk, hogy tovább haladva legfeljebb mekkora lehet a célfüggvény értéke és ha ez rosszabb mint az eddigi legjobb megoldásunk akkor levágjuk ezt a ágot, mivel javítás már onnan úgy sem jöhet
    - fa bejárás balról jobbra
  - a feladat egy becsléséből adódik, amikor is közelítjük, hogy milyen jó eredményt kaphatunk minden kisebb részproblémára
    - a becslést a relaxált feladat megoldása adja
    - ez a becslést (relaxált optimum) felülről korlátozza (bound) az egészértékű optimumot (maximalizálás esetén)

- algoritmus

1. **Relaxáció**

- oldjuk meg az aktuális feladat relaxáltját (relaxált feladat = eldobjuk az egészértékűségi feltételeket)
  - először az eredeti feladat relaxáltját, majd pedig későbbi iterációk során egy még nem lezárt részproblémának a relaxált feladatát

2. **Vizsgálat (Bounding / Korlátozás)**

- HA a **kapott megoldás egy egészérték**, akkor találtunk egy lehetséges megoldást ezt az ágat lezárjuk, és ha ez jobb mint az eddigi legjobb akkor lementjük, ha rosszabb nem kell lementeni, majd új részproblémát választunk melyre lefut az algoritmus
- HA a relaxált LP-nek **nincs megoldása**, akkor lezárjuk az adott részproblémát (ágat) és új részproblémát választunk melyre lefut az algoritmus
- HA **nem egészértékű a kapott megoldás és a célfüggvény értéke rosszabb**, mint egy már talált egész megoldásé, akkor ezt az ágat levágjuk (pruning), mert itt nem lehet jobb megoldás és új részproblémát választunk melyre lefut az algoritmus, amennyiben viszont **a célfüggvény értéke ígéretes** (jobb mint az eddigi legjobb megoldás) akkor tovább lépünk a következő lépésre (a szétválasztásra) ezzel a részproblémával

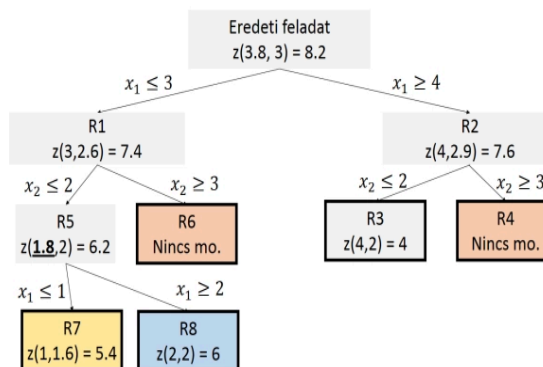
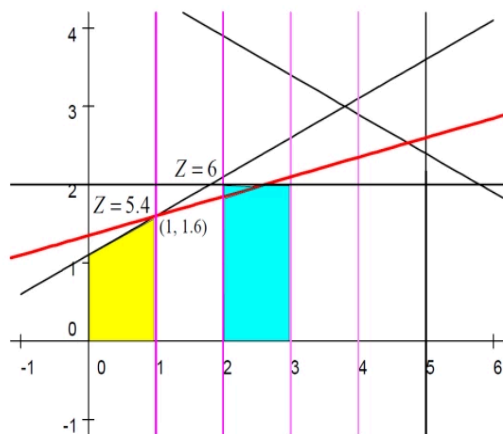
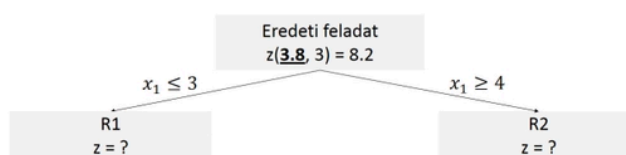
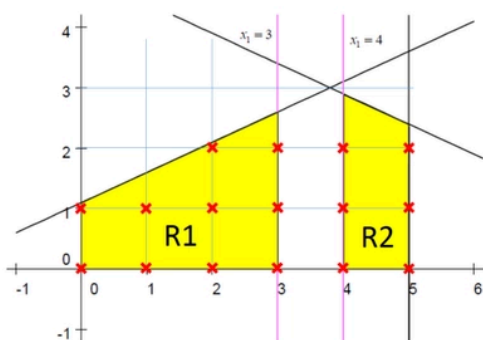
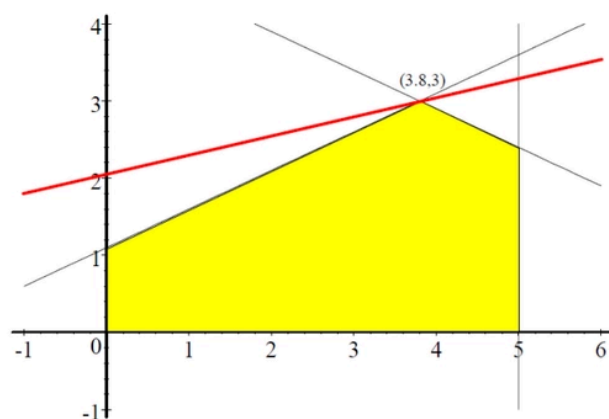
3. **Szétválasztás (Branching)**

- kiválasztjuk azt a változót mely tört értékű volt (pl:  $x_1 = 3.4$ ) és két új részproblémát generálunk belőle:
  - 1.ág: hozzáadjuk az  $x_1 \leq 3$  feltételt
  - 2.ág: hozzáadjuk az  $x_1 \geq 4$  feltételt
- ezzel kizárjuk a tört megoldást, de megtartjuk az összes lehetséges egész megoldást
- a következő ágat akár prioritási sorral (legígéretesebb), vagy veremmel (LIFO) vagy sorral (FIFO) oldhatjuk meg
- folyamatot addig ismételjük még minden ágat meg nem vizsgáltunk (minden lezárt) vagy le nem vágtuk
  - optimális megoldás a tárolt legjobb egész megoldás lesz
  - mikor lezárunk egy részproblémát (első 3 eset) akkor azt mondjuk, hogy teljesen felderített

- felépítése

$$\begin{array}{rclcl}
 -10x_1 & + & 20x_2 & \leq & 22 \\
 5x_1 & + & 10x_2 & \leq & 49 \\
 x_1 & & & \leq & 5 \\
 & & x_i & \geq & 0 \\
 \hline
 -x_1 & + & 4x_2 & \rightarrow & \text{max.}
 \end{array}$$

$$x_i \in \mathbb{Z}, i = 1, 2$$



ELFOGYTAK A LEZÁRATLAN ÁGAK! → VÉGE.

Az optimális egész megoldás az  $x^* = (2, 2)$  és  $z(x^*) = 6$ .

## Hátizsák feladat (Knapsack problem)

- ez egy IP (tiszta egészértékű programozási feladat) melyben csak 1 feltétel van
- **Feladat**
  - adott egy  $W$  kapacitású hátizsák és  $n$  darab tárgy. Minden tárgynak van egy  $w_i$  súlya és egy  $v_i$  értéke
  - a cél az összérték maximalizálása úgy, hogy az összsúly ne lépje túl a  $W$  kapacitást

**Matematikai modell:**

$$\text{Max } \sum v_i x_i$$

$$\text{Feltétel: } \sum w_i x_i \leq W, \quad x_i \in \{0, 1\}$$

- variánsai
  - ismétléses és ismétlés nélküli
  - töredékes (lehet törni a tárgyat és akkor az értéke is módosul) vagy egészértékű (nincs törés)
- megoldási módok
  - mohó algoritmus (greedy)
    - a tárgyakat fajlagos érték ( $v_i / w_i$ ) szerint rendezzük és a legjobbakat belerakjuk a hátizsákba
    - probléma, hogy nem garantál optimumot a 0-1 hátizsák feladatnál, mert üres helyek maradhatnak, amiket nem tudunk kihasználni
      - csak a folytonos (darabolható) változatnál működik
  - dinamikus programozás vagy branch and bound
    - ezekkel a módszerekkel garantáltan megtalálható az optimális megoldás

### Példa

Josie Camper egy kétnapos túrára készül. Négy tárgy van, amire szüksége lehet, de csak 1400 grammot tud ezekből elvinni. A tárgyak súlya, illetve haszna Josie szerint

| Tárgy     | Súly(100g) | Haszon | Relatív hasznosság | Sorrend |
|-----------|------------|--------|--------------------|---------|
| Tablet    | 5          | 16     | 3.2                | 1.      |
| Laptop    | 7          | 22     | 3.1                | 2.      |
| Okosteló  | 4          | 12     | 3                  | 3.      |
| Elemlámpa | 3          | 8      | 2.7                | 4.      |

### Matematikai modell

Legyen  $x_i = 1$  ha az  $i$ . tárgyat viszi,  $x_i = 0$  ha marad. Ekkor a feladat

$$\begin{aligned} \max \quad & z = 16x_1 + 22x_2 + 12x_3 + 8x_4 \\ \text{f.h.} \quad & 5x_1 + 7x_2 + 4x_3 + 3x_4 \leq 14 \\ & x_i \in \{0, 1\} \end{aligned}$$

- LP-lazítás (egy egészértékű programozási feladat relaxációját úgy kapjuk, hogy a változókra tett minden egészértékűségi vagy 0-1 (bináris) feltételt töröljük) könnyen számítható:
  - a relatív hasznosság szerint tesszük be a tárgyakat a hátizsákba, ami nem fér bele annak csak tört részét
    - relatív hasznosság:  $\text{haszon} / \text{súly}$ 
      - haszon másik nevén érték
  - példa alapján
    1. a tablet teljes egésze belekerül  $x_1 = 1$ , mivel relatív hasznossága a legnagyobb (3.2), marad még 1400 grammól 900 gramm szabad
    2. mivel a tablet teljes egészébe belekerült a következő legnagyobb relatív hasznossággal (3.1) a laptop bír és mivel még belefér az egész bele is kerül, tehát  $x_2 = 1$ , marad a hátizsákba még 200 gramm hely
    3. a telefon következik, de a súlya 400 gramm ezért csak fele fér bele,  $x_3 = 0.5$
    4. mivel elfogyott a hely ezért  $x_4 = 0$ , lesz tehát az elemlámpa már nem fér bele
    5. felírjuk a célfüggvény megoldását így  $x_1 = 1$ ,  $x_2 = 1$ ,  $x_3 = 0.5$ ,  $x_4 = 0$ , tehát a célfüggvény:  $z = 16 * x_1 + 22 * x_2 + 12 * x_3 + 8 * x_4 = 16 + 22 + 0.5 * 12 = 44$
  - az egyetlen tört változó szerint szétbontjuk 2 lehetséges részre
    - egyszer mikor az adott tört változó 0
      - példánk esetén pl  $x_3$  volt a tört és ezt vesszük  $x_3 = 0$ , ekkor mivel  $x_1$  és  $x_2$  már benne van megnézzük mi a következő és ez  $x_4$  lesz, aminek a 400 grammos helyre kéne beférni így  $x_4 = \frac{2}{3}$ , amivel ha kiszámítjuk a célfüggvény értékét:  $z = 42$ -t kapunk, ami rosszabb mint az eddigi legjobb
    - egyszer mikor az adott tört változó 1
- legrosszabb esetben  $2^n$  részfeladatot kell megoldani
  - tehát NP nehéz, nemdeterminisztikus polinomiális időben megoldható feladat
- mikor tört nem elfogadható (egészértékű feladat) akkor rosszabb, mivel  $2^M * n$  részfeladatot kell megoldani
  - $M$  a lehetséges egészek száma egy változóra
  - duál szimplex algoritmussal úgynevezett meleg indítással gyorsítható
    - ilyenkor az előző optimális szimplex táblából indulunk és a hozzávett nem lehetséges feltételből kell duál szimplex lépéseket végrehajtani
- fa bejárása lehet LIFO (Last In First Out) azaz mélységi bejárás, de FIFO (First In First Out), azaz szélességi bejárás, de akár más heurisztikán alapuló bejárás is

# HOZZÁRENDELÉSI FELADAT

## Munkák optimális kiosztása

- adott  $n$  számú dolgozó és ugyanennyi munka, egyes dolgozók a munkákat különböző költségekkel tudják végrehajtani
  - az  $i$ -edik munkás a  $j$ -edik munkát  $c_{ij}$  költséggel végzi el
- cél szétosztani az összes munkákat a dolgozók között úgy, hogy minden dolgozó egy munkát kapjon és a munkavégzés összköltsége minimális legyen (**optimális párosítás keresése**)
  - legyen  $x_{ij} = 1$ , ha az  $i$ -edik dolgozó a  $j$ -edik munkát végzi el, amennyiben nem ő végzi az adott munkát akkor  $x_{ij} = 0$
- a következő bináris lineáris programozási feladattal írhatjuk le:

$$(3.2.4) \quad \begin{aligned} \sum_{t=1}^n x_{it} &= 1 \quad (i = 1, \dots, n) \\ \sum_{t=1}^n x_{tj} &= 1 \quad (j = 1, \dots, n) \\ x_{ij} &\in \{0, 1\} \quad (i = 1, \dots, n; j = 1, \dots, n) \end{aligned}$$

$$\sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} = z \rightarrow \min$$

Valóban, minden  $i$ -re a  $\sum_{t=1}^n x_{it} = 1, x_{it} \in \{0, 1\} \quad (t = 1, \dots, n)$  feltétel biztosítja, hogy az  $i$ -edik munkás pontosan egy munkát végez el, és minden  $j$ -re a  $\sum_{t=1}^n x_{tj} = 1, x_{tj} \in \{0, 1\} \quad (t = 1, \dots, n)$  feltétel pedig azt eredményezi, hogy a  $j$ -edik munkát pontosan egy munkás végzi el. A megfelelő párosítás költsége  $c_{ij}$  definíciója alapján:  $\sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$ .

- a feladat lehetséges megoldásai olyan 0 és 1 elemekből álló **totálisan unimoduláris  $n * n$ -es mátrixok**, amelyek minden sora és oszlopa pontosan egy darab 1-et tartalmaz
  - totálisan unimoduláris (-1, 0, 1 értékeket vehetnek fel), ebből következik, hogy a változók és konstansok csak egész értékeket vehetnek fel, ebből következik, hogy a feladat reflexáltjának a megoldásával garantáltan optimális megoldást kapunk
  - minden ilyen típusú mátrix egy lehetséges megoldása a feladatnak ezekből  **$n!$  ( $n$  faktoriális) darab van**, ezért könnyen belátható, hogy különböző célfüggvényű hozzárendelési feladatokban amiben  $n$  megegyezik a lehetséges megoldások halmaza is megegyezik
    - ebből következik, hogy **a feladatot egyértelműen meghatározza a  $c_{ij}$  súlymátrix (költségmátrix)**

## Magyar módszer

- Kőnig Dénes, Egerváry Jenő, Harold W. Kuhn
- algoritmus a hozzárendelési feladat gyors, hatékony és optimális megoldásához
- $C$  egy  $n * n$ -es bináris mátrix
  - ha úgy fedjük le  $C$  bizonyos sorait és oszlopait, hogy az összes 1-est lefedik akkor fedőrendszeréről beszélünk
    - legkevesebb sort és oszlopot lefedő fedést, minimális fedésnek hívjuk
  - válasszunk  $C$ -ből maximális számú egyest, úgy, hogy egy sorból vagy oszlopból legfeljebb egy darab 1-est választunk, ezeknek száma megegyezik a minimális fedésben szereplő sor és oszlop fedések számával
  - ezek után márcsak a következő probléma megoldása maradt hátra: hogyan lehet az általános hozzárendelési feladatot olyan hozzárendelési feladatra visszavezetni, amelyben a költségmátrix bináris
- Páros gráfok párosítási problémái
  - adott:
    - $(G; A, B)$  kiegyensúlyozott páros gráf:  $|A| = |B|$ 
      - feltesszük, hogy teljes, ezért a tiltásokat (-végtelen értékeket) nem engedünk
      - akkor páros egy gráf ha  $V(G) = A \cup B$ , ahol  $A \cap B = \emptyset$ , és minden él  $A$  és  $B$  között húzódik
    - egy  $w$  élsúlyozás, amit egy  $R_+^{A * F}$  négyzetes mátrixsal írhatunk le
  - cél egy  $M$  maximális súlyú teljes párosítást találni
    - $M$  azon élekből áll ahol  $x_{ij}^* = 1$ , azaz optimális megoldást jelöl
    - ezt LP feladattal az ábrán látható módon kell megkapni:

$$\max \sum_{j \in B} \sum_{i \in A} x_{ij}$$

$$\sum_{j: (i,j) \in E(G)} x_{ij} \leq 1 \quad (i \in A)$$

$$\sum_{i: (i,j) \in E(G)} x_{ij} \leq 1 \quad (j \in B)$$

$$x_{ij} \leq 0 \quad (i \in A, j \in B),$$

- **Kőnig Dénes tétele:** ha  $|A| = |B| = n$  (kiegyensúlyozott páros gráf) és minden  $v \in V(G)$  pontfokszáma  $d(v) = k \geq 1$  esetén van teljes párosítás a  $G$  gráfnak
  - bizonyítása:
    - készítsünk egy transshipment problémát a  $G$  gráfhoz úgy, hogy  $A$  pontjaihoz  $-1$ -et, míg  $B$  pontjaihoz  $1$ -et rendelünk, az éleket pedig  $A$ -ból  $B$ -be irányítjuk (élek költségével nem törődünk)
    - ennek az új problémának van lehetséges  $x$  megoldása  $x_e = 1/k$  minden  $e$  élre



- integrálási tétel miatt, így van  $x^*$  lehetsége egész megoldása ( $x_e^* \in \{0, 1\}$  minden  $e$  élre)
  - $A$  minden pontjából pontosan egy él vezet ki, melyekhez tartozó változó értéke 1, és ezek az élek egy  $M$  teljes párosítást adnak
- menete
  - Előkészítő rész
    - soronkénti redukció
      - $C$  mátrix  $i$ -edik sorából vonjuk ki az  $i$ -edik sor elemeinek a minimumát, az előálló mátrixot  $C$ (felülvonás) jelölje
        - minden sorból a sor legkisebb elemét vonjuk ki
    - oszloponkénti redukció
      - $C$ (felülvonás) mátrix  $j$ -edik oszlopának elemeiből vonjuk ki az illető oszlop elemeinek a minimumát és a kapott mátrixot jelölje a  $C^{(0)}$ 
        - minden oszlopból az oszlop legkisebb elemét vonjuk ki
    - $C^{(0)}$ -ban jelöljük ki egy független 0-rendszert oszloponként haladva, egy adott oszlopban mindig a legkisebb sorindexű 0-t vegyük fel a független 0-rendszerbe (0\*-al jelöljük)
    - $r$  értéke legyen nulla és menjünk a következő részre (iterációs eljárás)
  - Iterációs rész ( $r$ . iterációs)
    1. lépés
      - HA a  $C^{(r)}$ -ben kijelölt független 0-rendszer elemeinek száma  $n$  akkor vége az eljárásnak (0\*-ok száma =  $n$ )
        - optimális megoldás
      - HA nem  $n$  az elemek száma, akkor pedig kössük le  $C^{(r)}$ -ben a 0\*-okat tartalmazó oszlopokat és folytassuk a következő lépéssel
    2. lépés
      - keressünk soronként haladva szabad 0-t
      - HA nincs szabad 0 akkor az 5. lépés következik
      - HA találunk szabad 0-t akkor vizsgáljuk meg az adott sort ahol találtuk
        - HA ez a sor nem tartalmaz 0\*-ot, akkor a 4. lépés jön
        - HA tartalmaz a sor 0\*-ot, akkor a 3. lépéssel folytatjuk
    3. lépés
      - az adott szabad 0-t lássuk el „,”-vel, kössük le a sorát és szüntessük meg a sorában lévő 0\* oszlopának lekötését, majd folytassuk az eljárást a 2. lépéssel
    4. lépés
      - az adott szabad 0-t lássuk el ’-vel és ebből a 0’-ból kiindulva képezzünk láncot a következő szerint:
        - minden láncbeli 0`-re a sorában lévő 0`-vel folytatódik a

láncképzés véget ér

- mikor a láncképzés véget ér a  $C^{(r+1)}$  a jelölések nélküli aktuális  $C^{(r)}$  mátrix legyen és lássuk el „\*”-gal a  $c_{ij}^{(r)}$  elemet:
  - HA  $c_{ij}^{(r)} = 0^*$  és  $c_{ij}^{(r)}$  nem szerepel a láncképzésben
  - VAGY HA  $c_{ij}^{(r)} = 0'$  és  $c_{ij}^{(r)}$  eleme a láncképzésnek
- majd  $r$  értékét növeljük 1-gyel és folytassuk az eljárást a következő iterációval

## 5. lépés

- képezzük a szabad elemek minimumát, majd ezt vonjuk ki az összes szabad elemből és adjuk hozzá a kétszer kötött (sorok és oszlopok is le vannak kötve) elemekhez
  - az átalakított mátrixot mentjük a  $C^{(r)}$ -be és folytassuk az eljárást a 2. lépéssel
- Amennyiben vannak tiltott hozzárendelések, amelyeket nem választhatunk ki, a feladat visszavezethető a hozzárendelési feladatra, a tiltott helyeken végtelen értékre módosítva a költségmátrixot

$$C = \begin{pmatrix} 4 & 5 & 3 & 2 & 3 \\ 3 & 2 & 4 & 3 & 4 \\ 3 & 3 & 4 & 4 & 3 \\ 2 & 4 & 3 & 2 & 4 \\ 2 & 1 & 3 & 4 & 3 \end{pmatrix}$$

A sorminimumok a következők: 2, 2, 3, 2, 1. Ezeket rendre kivonva az első, ..., ötödik sor elemeiből, az alábbi  $\bar{C}$  mátrixhoz jutunk.

$$\bar{C} = \begin{pmatrix} 2 & 3 & 1 & 0 & 1 \\ 1 & 0 & 2 & 1 & 2 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 2 & 1 & 0 & 2 \\ 1 & 0 & 2 & 3 & 2 \end{pmatrix}$$

A  $\bar{C}$  mátrix oszlopaira képezve a minimumokat, a következő értékeket kapjuk: 0, 0, 1, 0, 0. Rendre kivonva ezeket a megfelelő oszlopok elemeiből (most csak a harmadik oszlop elemeiből kell kivonnunk 1-et), az alábbi  $C^{(0)}$  mátrixot kapjuk.

$$C^{(0)} = \begin{pmatrix} 2 & 3 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 2 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 & 2 \\ 1 & 0 & 1 & 3 & 2 \end{pmatrix}$$

Most jelöljük ki  $C^{(0)}$ -ban egy független 0-rendszert oszlopfolytonosan. Az első oszlop első 0 eleme  $c_{31}^{(0)}$ . Vegyük fel ezt az elemet a független 0-rendszerbe, azaz lássuk el „\*”-gal, majd térjünk át a második oszlop 0 elemeinek a vizsgálatára. Ebben az oszlopban az első 0 elem  $c_{22}^{(0)}$ . Ennek a sora nem tartalmaz 0\* elemet, így bővíthetjük vele a rendszert, és áttérünk a harmadik oszlopra. Itt az első 0 elem  $c_{13}^{(0)}$ , amellyel szintén bővíthető a független 0-rendszer. A negyedik oszlop első 0 eleme  $c_{14}^{(0)}$ , amelynek a sora tartalmaz 0\*-ot, így ezzel nem bővíthető a rendszer. A negyedik oszlop második 0 eleme  $c_{44}^{(0)}$ , amivel ismét bővíthető a független 0-rendszer. Végül az utolsó oszlop első 0 eleme  $c_{35}^{(0)}$ . A harmadik sor már tartalmaz 0\*-ot, így ezzel az elemmel nem bővíthetünk. Mivel az utolsó oszlop több 0 elemet nem tartalmaz, ezért a független 0-rendszer kijelölését, és egyben az eljárás előkészítő részét is befejeztük. A kijelölt független 0-rendszer a következő:

$$C^{(0)} = \begin{pmatrix} 2 & 3 & 0^* & 0 & 1 \\ 1 & 0^* & 1 & 1 & 2 \\ 0^* & 0 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0^* & 2 \\ 1 & 0 & 1 & 3 & 2 \end{pmatrix}$$

Térjünk rá ezek után az iterációs rész végrehajtására. Mivel a független 0-rendszer elemeinek a száma 4, ezért kössünk le minden olyan oszlopot, amely  $0^*$ -ot tartalmaz.

$$C^{(0)} = \begin{matrix} & + & + & + & + \\ \begin{pmatrix} 2 & 3 & 0^* & 0 & 1 \\ 1 & 0^* & 1 & 1 & 2 \\ 0^* & 0 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0^* & 2 \\ 1 & 0 & 1 & 3 & 2 \end{pmatrix} \end{matrix}$$

Sorfolytonosan szabad 0-t keresve,  $c_{35}^{(0)}$  az első szabad 0. A harmadik sor tartalmaz  $0^*$ -ot, így a harmadik lépés szerint  $c_{35}^{(0)}$ -t ellátjuk vesszővel, a harmadik sort lekötjük, majd a sorban lévő  $0^*$  oszlopát (első oszlop) feloldjuk. A kapott mátrix a következő:

$$C^{(0)} = \begin{matrix} & + & + & + \\ \begin{pmatrix} 2 & 3 & 0^* & 0 & 1 \\ 1 & 0^* & 1 & 1 & 2 \\ 0^* & 0 & 0 & 1 & 0' \\ 0 & 2 & 0 & 0^* & 2 \\ 1 & 0 & 1 & 3 & 2 \end{pmatrix} \end{matrix} +$$

Ezt követően a 2. lépéssel, sorfolytonos 0 kereséssel folytatjuk az eljárást. Most  $c_{41}^{(0)}$  az első szabad 0. A negyedik sor tartalmaz  $0^*$ -ot, ezért a 3. lépés alapján  $c_{41}^{(0)}$ -t ellátjuk vesszővel, a negyedik sort lekötjük, és a negyedik oszlopot feloldjuk.

$$C^{(0)} = \begin{matrix} & + & + \\ \begin{pmatrix} 2 & 3 & 0^* & 0 & 1 \\ 1 & 0^* & 1 & 1 & 2 \\ 0^* & 0 & 0 & 1 & 0' \\ 0' & 2 & 0 & 0^* & 2 \\ 1 & 0 & 1 & 3 & 2 \end{pmatrix} \end{matrix} +$$

Ismét rátérve a 2. lépésre,  $c_{14}^{(0)}$  az első szabad 0. Az első sor tartalmaz  $0^*$ -ot, ezért  $c_{14}^{(0)}$ -t vesszővel látjuk el, az első sort lekötjük, és a harmadik oszlopot feloldjuk.

$$C^{(0)} = \begin{matrix} & + \\ \begin{pmatrix} 2 & 3 & 0^* & 0' & 1 \\ 1 & 0^* & 1 & 1 & 2 \\ 0^* & 0 & 0 & 1 & 0' \\ 0' & 2 & 0 & 0^* & 2 \\ 1 & 0 & 1 & 3 & 2 \end{pmatrix} \end{matrix} +$$

Újra a 2. lépéssel folytatva az eljárást, azt kapjuk, hogy nincs szabad 0. Ekkor az 5. lépés következik. Abból a célból, hogy az 5. lépés végrehajtását megkönnyítsük, fedjük le a kötött sorokat és oszlopokat. Így a le nem fedett elemek lesznek pontosan a szabad elemek. Ezek minimuma 1. Ezt a minimumot kivonva a szabad elemekből és hozzáadva a kétszer fedett elemekhez, a következő mátrixhoz jutunk.

$$C^{(0)} = \begin{matrix} & + \\ \begin{pmatrix} 2 & 4 & 0^* & 0' & 1 \\ 0 & 0^* & 0 & 0 & 1 \\ 0^* & 1 & 0 & 1 & 0' \\ 0' & 3 & 0 & 0^* & 2 \\ 0 & 0 & 0 & 2 & 1 \end{pmatrix} \end{matrix} +$$

Most ismét a 2. lépéssel kell folytatnunk az eljárást. Sorfolytonosan  $c_{21}^{(0)}$  az első szabad 0. Mivel  $c_{21}^{(0)}$  sora tartalmaz 0\*-ot, ezért  $c_{21}^{(0)}$ -t ellátjuk vesszővel, a sorát lekötjük, a második oszlopot feloldjuk.

$$C^{(0)} = \begin{pmatrix} 2 & 4 & 0^* & 0' & 1 \\ 0' & 0^* & 0 & 0 & 1 \\ 0^* & 1 & 0 & 1 & 0' \\ 0' & 3 & 0 & 0^* & 2 \\ 0 & 0 & 0 & 2 & 1 \end{pmatrix} +$$

Újra szabad 0-t keresve,  $c_{51}^{(0)}$  lesz az első szabad 0. Mivel  $c_{51}^{(0)}$  sora nem tartalmaz 0\*-ot, ezért a 4. lépéssel folytatódik az eljárás.  $c_{51}^{(0)}$ -t ellátjuk vesszővel, és ez lesz a lánc kezdő eleme. A lánc következő eleme az első oszlopban lévő 0\*, azaz  $c_{31}^{(0)}$ . Ezt az elemet a láncban a harmadik sorban elhelyezkedő 0', azaz  $c_{35}^{(0)}$  követi. Mivel az ötödik oszlop nem tartalmaz 0\*-ot, ezért  $c_{35}^{(0)}$ -val a lánc véget ér. Ezek után törölve a jelöléseket, és „\*”-gal ellátva a láncon kívüli 0\*-oknak megfelelő elemeket ( $c_{13}^{(0)}, c_{22}^{(0)}, c_{44}^{(0)}$ ), valamint „\*”-gal ellátva a láncbeli 0'-knek megfelelő elemeket ( $c_{35}^{(0)}, c_{51}^{(0)}$ ), az alábbi  $C^{(1)}$  mátrixot kapjuk.

$$C^{(1)} = \begin{pmatrix} 2 & 4 & 0^* & 0 & 1 \\ 0 & 0^* & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0^* \\ 0 & 3 & 0 & 0^* & 2 \\ 0^* & 0 & 0 & 2 & 1 \end{pmatrix}$$

Mivel  $C^{(1)}$ -ben egy 5-elemű független 0-rendszer van kijelölve, ezért az eljárás véget ér. Az előzőek alapján tudjuk, hogy ebben az esetben egy optimális megoldás az alábbi  $\bar{X}$  mátrix:

$$\bar{X} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

A végrehajtott eljárással kapcsolódva, fontosnak tartjuk megjegyezni a következőket.

A szükséges iterációk  $k$  száma  $C$ -től függően a  $0, 1, \dots, n-1$  értékek bármelyike lehet. Például konstruálható olyan  $5 \times 5$ -ös  $C$  mátrix, amelyre alkalmazva az eljárást,  $C^{(0)}, C^{(1)}, C^{(2)}, C^{(3)}, C^{(4)}$  az előálló mátrixsorozat.

A láncképzés során előfordulhat, hogy a lánc egyetlen 0' elemből, a kezdő elemből áll. Ilyenkor *elfajuló láncról* beszélünk, amely úgy interpretálható, hogy

a benne szereplő 0'-vel közvetlenül bővíthető az előzőekben előállított független 0-rendszer.

Az algoritmus fenti végrehajtásában az egyes lépéseket szándékosan túlrészleteztük.  $C^{(0)}$  ismételt kiírása nélkül kijelölhető  $C^{(0)}$ -ban a független 0-rendszer, lekötethetők az oszlopok, és végrehajthatók a sorkötések valamint az oszlopfeloldások. Ez utóbbit a „+” jel bekarikázásával jelöljük. A mátrix ismételt kiírása csak az 5. lépés végrehajtása után szükséges. Az elmondottaknak megfelelően végrehajtva az eljárást, az alábbi mátrixokhoz jutunk:

$$\begin{pmatrix} 4 & 5 & 3 & 2 & 3 \\ 3 & 2 & 4 & 3 & 4 \\ 3 & 3 & 4 & 4 & 3 \\ 2 & 4 & 3 & 2 & 4 \\ 2 & 1 & 3 & 4 & 3 \end{pmatrix} \oplus \begin{pmatrix} 2 & 3 & 1 & 0 & 1 \\ 1 & 0 & 2 & 1 & 2 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 2 & 1 & 0 & 2 \\ 1 & 0 & 2 & 3 & 2 \end{pmatrix} \oplus \begin{pmatrix} 2 & 3 & 0^* & 0' & 1 \\ 1 & 0^* & 1 & 1 & 2 \\ 0^* & 0 & 0 & 1 & 0' \\ 0' & 2 & 0 & 0^* & 2 \\ 1 & 0 & 1 & 3 & 2 \end{pmatrix} +$$

$$\oplus \begin{pmatrix} 2 & 4 & 0^* & 0' & 1 \\ 0' & 0^* & 0 & 0 & 1 \\ 0^* & 1 & 0 & 1 & 0' \\ 0' & 3 & 0 & 0^* & 2 \\ 0' & 0 & 0 & 2 & 1 \end{pmatrix} + \begin{pmatrix} 2 & 4 & 0^* & 0 & 1 \\ 0 & 0^* & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0^* \\ 0 & 3 & 0 & 0^* & 2 \\ 0^* & 0 & 0 & 2 & 1 \end{pmatrix}$$